Studio Nx

Backend(FastAPI/Python) 06 — get_current_user()

2026-10-07

Backend(FastAPI/Python) 06 — `get_current_user()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통 — 로그인한 사용자 찾기 · PRD 01 FR-105 · FR-108
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	05 `decode_access_token()` · 07 `get_current_user_optional()`

1. 하는 일

요청 헤더의 Access Token 으로 **요청한 회원을 DB 에서 찾아** 라우터에 넘긴다. 토큰이 없거나, 틀렸거나, 회원이 활동 중이 아니면 401 오류를 던진다.

`Authorization: Bearer eyJhbGciOi…` → `User(user_id=7, …)`

2. 언제 쓰나

로그인이 필요한 API 가 함수 인자에 `user: CurrentUser` 라고 적으면, FastAPI 가 라우터보다 **먼저** 이 함수를 부른다. 라우터는 검사가 끝난 회원만 받는다.

```
@router.get("/users/me")
def read_me(user: CurrentUser) -> MeResponse:   # user = 이 함수가 돌려준 회원
    ...
```

이 함수를 쓰는 API (지금 뼈대 기준)	화면
`GET /users/me` · `PATCH /users/me` · 프로필 사진 · 비밀번호 변경 · 탈퇴	userInfo.html
`GET /me/dashboard`	myPage.html
`GET · POST · PATCH · DELETE /projects…`	myProject.html · studioNx.html
쓰지 않음: `POST /inquiries`	로그인 선택 → 07 `get_current_user_optional()`

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard` + `Authorization: Bearer eyJ…` (JS 02)
② `bearer_scheme`	헤더에서 `eyJ…` 만 꺼냄
③ `get_db`	DB 세션을 엶
④ **이 함수**	토큰 검사(05) → 7 → DB 에서 7번 회원 → 상태 확인
⑤ 라우터	`user` 로 7번 회원을 받아 대시보드를 만듦
실패하면	라우터는 실행되지 않음. 401 → 브라우저 `api.js` 가 재발급 후 한 번 다시 보냄

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import 이다. **`AppError` · `ErrorCode` · `decode_access_token` 두 줄을 이번에 추가했다.**

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 덧붙임 정보를 다는 도구	`Bearer` · `Db` · `CurrentUser` 별칭
`Depends`	"이 값은 이 함수로 만들어 넣어라" 표시	별칭 안
`HTTPBearer` · `HTTPAuthorizationCredentials`	`Authorization: Bearer …` 헤더 읽기 · 그 결과	`bearer_scheme` · `credentials`
`Session`	DB 세션 타입 (SQLAlchemy)	`Db`
`AppError` · `ErrorCode`	01 의 오류 클래스 · 코드	401
`decode_access_token`	05 의 토큰 검사	2단계
`get_db`	요청마다 DB 세션을 열고 닫는 함수	`Db`
`User`	users 테이블 모델	`db.get(User, …)`

함수 (이미 있던 별칭 포함)

```
bearer_scheme = HTTPBearer(auto_error=False)
Bearer = Annotated[HTTPAuthorizationCredentials | None, Depends(bearer_scheme)]
Db = Annotated[Session, Depends(get_db)]


def get_current_user(
    credentials: Bearer,
    db: Db,
) -> User:
    if credentials is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_access_token(credentials.credentials)
    user = db.get(User, user_id)
    if user is None or user.status != "ACTIVE":
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user


CurrentUser = Annotated[User, Depends(get_current_user)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀
`app.…`	우리 프로젝트(`studionx/backend/app/`)의 파일. `app.api.deps` = `app/api/deps.py`

식	뜻
`Depends(함수)`	FastAPI 가 요청마다 그 함수를 먼저 실행해 결과를 인자로 넣어 줌 ("의존성")
`Annotated[타입, Depends(…)]`	타입과 `Depends` 를 한 이름으로 묶음. `user: CurrentUser` 한 번만 쓰면 됨
`HTTPBearer(auto_error=False)`	헤더가 없거나 `Bearer` 로 시작하지 않으면 **오류 대신** **`None`** 을 넣어 줌. 기본값(`True`)이면 FastAPI 가 그 자리에서 401 오류를 던짐
`credentials.credentials`	`Bearer eyJ…` 에서 `eyJ…` 부분 (`credentials.scheme` 은 `"Bearer"`)
`db.get(User, 7)`	기본 키(`user_id`)가 7 인 행. 없으면 `None`
`A is None or B`	`A` 가 참이면 `B` 를 보지 않음 — `None.status` 오류가 나지 않음

요청	결과
헤더 없음	401 "Please sign in."
`Authorization: Basic …`	401 "Please sign in." (`Bearer` 가 아님 → `None`)
토큰이 틀림 · 만료	401 "Please sign in again." (05 가 던짐)
토큰은 맞는데 회원 없음	401 "Please sign in again."
회원 `status` 가 `SUSPENDED` · `DELETED`	401 "Please sign in again."
회원 `status` 가 `ACTIVE`	그 회원 (`User`)

4. 흐름

1. 헤더가 없으면 401 "Please sign in."
2. 토큰을 검사해 `user_id` 를 얻는다 (05). 실패하면 401.
3. DB 에서 그 회원을 찾는다.
4. 없거나 `ACTIVE` 가 아니면 401 "Please sign in again."
5. 회원을 돌려준다.

5. 왜 이렇게

선택	이유
토큰만 믿지 않고 DB 를 한 번 더 확인	Access Token 은 30분 동안 유효하다. 그 사이 탈퇴(`DELETED`)하거나 정지(`SUSPENDED`)된 회원의 토큰을 막는다 (FR-108)
없는 회원 · 정지 · 탈퇴를 같은 401 · 같은 문구로	05 와 같은 이유. 이유를 알려 주면 "이 번호의 회원은 탈퇴했다" 같은 정보가 샌다
403 이 아니라 401	프론트 `api.js` 는 401 `UNAUTHORIZED` 일 때만 재발급 → 실패하면 로그인 화면으로 보낸다 (JS 02). 정지 · 탈퇴 회원은 재발급도 거절되어(P1-05) 결국 로그인 화면으로 간다
문구 두 가지	헤더 없음은 처음부터 로그인 안 한 것 → "Please sign in." 토큰이 있는데 안 됨 → "Please sign in again."

선택	이유
`auto_error=False` 로 직접 401	같은 `Bearer` 를 07 `get_current_user_optional()` 도 쓴다. 07 은 헤더가 없을 때 오류가 아니라 `None`(손님)으로 처리해야 하므로, 헤더 읽기는 오류를 내지 않게 두고 각 함수가 판단한다
`status != "ACTIVE"` 를 글자로 비교	users 모델의 `status` 칸은 이 작업의 다음 순서(모델 컬럼)에서 만든다. 그때 `StrEnum` 으로 만들면 `UserStatus.ACTIVE == "ACTIVE"` 가 참이라 이 코드는 그대로 동작한다

6. 시험 (`tests/test_deps.py`, 새로 만듦 · 7개)

users 모델의 컬럼(`status` 등)은 아직 없어서, 진짜 DB 대신 **가짜 세션(`FakeDb`)**을 넣었다. `FakeDb` 는 `db.get(User, id)` 만 흉내 내고, 받은 인자를 기록한다. 시험용 앱에 `GET /whoami` (`user: CurrentUser` → `{"user_id": …}`)를 만들어 요청했다.

경우	결과
정상 토큰 + `ACTIVE` 회원 7	200 `{"user_id": 7}` · DB 에 `(User, 7)` 로 한 번 물어봄
헤더 없음	401 "Please sign in." · DB 에 묻지 않음
`Authorization: Basic …`	401 "Please sign in."
틀린 토큰	401 "Please sign in again." · DB 에 묻지 않음
토큰 회원 99 가 DB 에 없음	401 "Please sign in again."
`SUSPENDED` · `DELETED` 회원 (2개)	401 "Please sign in again."

`app.dependency_overrides[get_db] = …`: 시험에서 `get_db` 대신 다른 함수를 쓰게 하는 FastAPI 기능이다.

전체 시험 42개 통과, `ruff` 검사 통과.